

WEEK 7 PRACTICAL

QUESTION 1

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Aim

To write a C program that displays the memory addresses of variables belonging to the **code**, **global**, **static**, **heap**, and **stack** segments and understand the Linux process address-space layout.

Program

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

static int static_global_var = 200;

void sample_function() {
    printf("Inside sample function\n");
}

int main() {
    int stack_var = 300;

    static int static_local_var = 400;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL) {
        perror("malloc");
        return 1;
    }
    *heap_var = 500;
    printf("Memory Addresses:\n\n");

    printf("Code (function)           : %p\n", (void *)sample_function);
    printf("Global variable                : %p\n", (void *)&global_var);
    printf("Static global variable          : %p\n", (void *)&static_global_var);
    printf("Static local variable          : %p\n", (void *)&static_local_var);
    printf("Heap variable                   : %p\n", (void *)heap_var);
    printf("Stack variable                  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Memory Addresses:

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ mkdir -p Week7
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ cd Week7
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ nano memory_segments.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ gcc memory_segments.c -o memory_segments
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ ./memory_segments
```

Memory Addresses:

```
Code (function)      : 0x5dd0f008a1a0
Global variable      : 0x5dd0f9991818
Static global variable : 0x5dd0f9991814
Static local variable : 0x5dd0f9991810
Heap variable        : 0x5dd0fe82b818
Stack variable       : 0x7ffe8c55504c
```

Observation

The program displays the addresses of variables located in different parts of the process address space.

Memory Segment	Example	Purpose
Code / Text	sample_function	Contains executable program instructions
Global / Data	global_var	Stores initialized global variables
Static	static_global_var, static_local_var	Static variables have program-long lifetime and are typically stored in the data or BSS segment depending on initialization.
Heap	heap_var	Used for dynamically allocated memory using malloc()
Stack	stack_var	Stores local variables and function-call information

The addresses demonstrate that different types of data are maintained in different regions of the Linux process address space.

Result

The program was successfully implemented to print the addresses of **code, global, static, heap, and stack variables**. The Linux process memory layout was analyzed based on the addresses obtained.

QUESTION 2

Make use of the `/proc/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Aim

To study the memory organization of a running Linux process using the `/proc/<PID>/maps` file and memory-analysis commands, and understand virtual memory mappings.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int local_var = 200;

    printf("Process ID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Stack variable address : %p\n", (void *)&local_var);

    printf("\nProcess is running...\n");
    printf("Open another terminal and execute:\n");
    printf("cat /proc/%d/maps\n", getpid());

    printf("\nPress Enter to terminate...\n");
    getchar();

    return 0;
}
```

Output

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ nano memory_maps.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ gcc memory_maps.c -o memory_maps
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ ./memory_maps
Process ID: 1722
Global variable address: 0x5ee50db7c010
Stack variable address : 0x7ffd2f64f7d4

Process is running...
Open another terminal and execute:
cat /proc/1722/maps

Press Enter to terminate...
```

Useful Memory Analysis Commands

To examine the process mappings:

```
cat /proc/1722/maps
```

Sample /proc/1722/maps Output

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ cat /proc/1722/maps
5ee5041f0000-5ee5041d7000 r--p 00000000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
5ee5041d7000-5ee5041b7000 r-xp 00001000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
5ee5041b7000-5ee5041b4000 r--p 00021000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
5ee5041b4000-5ee5041b5000 r--p 00021000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
5ee5041b5000-5ee5041b6000 rw-p 00022000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
7fac3a152000-7fac3a153000 rw-p 00000000 00:00 0 [heap]
7fac7ba76000-7fac7ba78000 r--p 001ce000 16:30 17677 /usr/lib/x86_64-linux-gnu/libc.so.6
7fac7ba78000-7fac7bbd0000 r-xp 001f0000 16:30 17677 /usr/lib/x86_64-linux-gnu/libc.so.6
7fac7bbd0000-7fac7bc23000 r--p 00368000 16:30 17677 /usr/lib/x86_64-linux-gnu/libc.so.6
7fac7bc23000-7fac7bc25000 r--p 0037b000 16:30 17677 /usr/lib/x86_64-linux-gnu/libc.so.6
7fac7bc25000-7fac7bc27000 rw-p 0037d000 16:30 17677 /usr/lib/x86_64-linux-gnu/libc.so.6
7fac7bc27000-7fac7bc34000 rw-p 00000000 00:00 0
7fac7bc34000-7fac7bc35000 r--p 00000000 00:00 0
7fac7bc35000-7fac7bc36000 rw-p 00000000 00:00 0
7fac7bc36000-7fac7bc39000 rw-p 00000000 00:00 0 [vvar]
7fac7bc39000-7fac7bc3b000 r--p 00000000 00:00 0 [vvar_vclock]
7fac7bc3b000-7fac7bc3d000 r-xp 00000000 00:00 0 [vdso]
7fac7bc3d000-7fac7bc3f000 r--p 00000000 16:30 15644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fac7bc3f000-7fac7bc64000 r-xp 00002000 16:30 15644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fac7bc64000-7fac7bc6e000 r--p 00027000 16:30 15644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fac7bc6e000-7fac7bc70000 r--p 00031000 16:30 15644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fac7bc70000-7fac7bc72000 rw-p 00033000 16:30 15644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffd22f65000-7ffd22f86000 rw-p 00000000 00:00 0
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ [stack]
```

To view memory statistics:

cat /proc/1722/status

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ cat /proc/1722/status
Name:   memory_maps
Umask:  0022
State:  S (sleeping)
Tgid:   1722
Ngid:   0
Pid:    1722
PPid:   793
TracerPid: 0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 1000
NSTgid: 1722
NSpid:  1722
NSpgid: 1722
NSSid:  793
Kthread: 0
VmPeak:  2768 kB
VmSize:  2768 kB
VmLck:   0 kB
```

To view detailed memory information:

cat /proc/1722/smaps

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week7$ cat /proc/1722/smaps
5ee5041f0000-5ee5041d7000 r--p 00000000 16:30 16251 /home/jhanasri/OSSP/Week7/memory_maps
Size: 4 kB
KernelPageSize: 4 kB
MMUPageSize: 4 kB
Rss: 4 kB
Pss: 4 kB
Pss_Dirty: 0 kB
Shared_Clean: 0 kB
Shared_Dirty: 0 kB
Private_Clean: 4 kB
Private_Dirty: 0 kB
Referenced: 4 kB
Anonymous: 0 kB
KSM: 0 kB
LazyFree: 0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap: 0 kB
SwapPss: 0 kB
Locked: 0 kB
THPeligible: 0
```

You can also use:

pmap 1722

Observation

The `/proc/<PID>/maps` file shows the **virtual memory regions** assigned to a running process.

Common mappings include:

- **Executable/text region** — contains the program's machine instructions.
- **Read-only region** — contains read-only data and other protected mappings.
- **Data region** — contains initialized and writable program data.
- **Heap** — used for dynamically allocated memory.
- **Shared libraries** — libraries such as the C library are mapped into the process's virtual address space.
- **Stack** — contains local variables and function-call information.
- **VDSO/VVAR** — special kernel-provided mappings used for certain system operations and timing-related functionality.

The permissions shown in `/proc/<PID>/maps` describe what operations are allowed:

r = read

w = write

x = execute

p = private mapping

s = shared mapping

Result

The memory organization of a running Linux process was successfully studied using `/proc/<PID>/maps`, `/proc/<PID>/status`, `/proc/<PID>/smaps`, and `pmap`. The experiment demonstrated how **virtual memory mappings** divide a process's address space into code, data, heap, stack, libraries, and other regions.